

Java NIO (File IO) - Part1

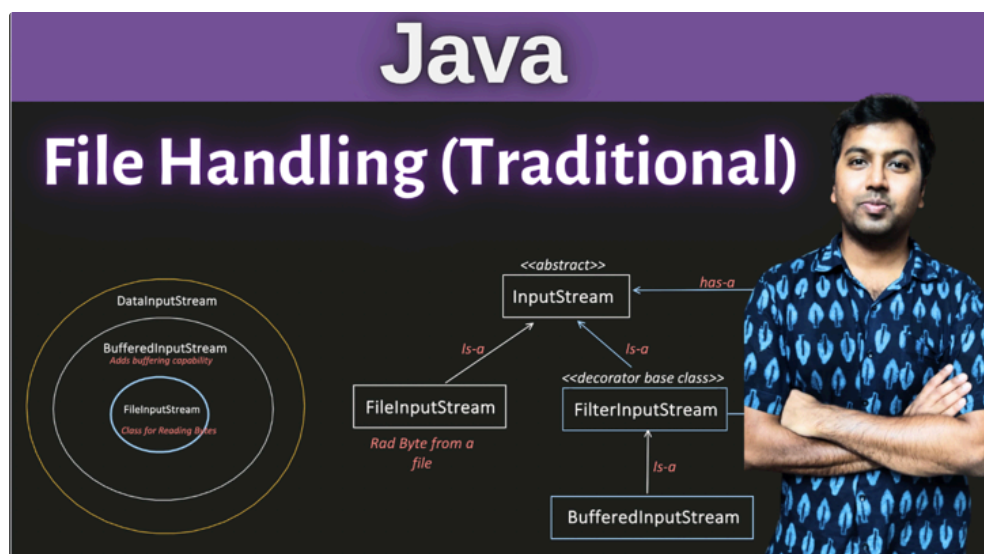
NIO (New Input/Output):

It is divided in 2 parts:-

1. File IO
2. Network IO

will focus on File IO part in this.

Question: When we already have File Handling via Stream, Why we need NIO (File IO)?



Check Traditional File handling topic, if there is any doubt with File Handling via Stream

Some of the Advantages which NIO (File IO) brings over Traditional File handling via Streams are:

1. Bi-directional:

Traditional File handling(Streams) are Uni-directional:

- FileInputStream: used for Reading from the file.
- FileOutputStream: used for Write to the file.

NIO (Channel) are Bi-directional:

- FileChannel: used for both Reading and Writing.

2. Random Access:

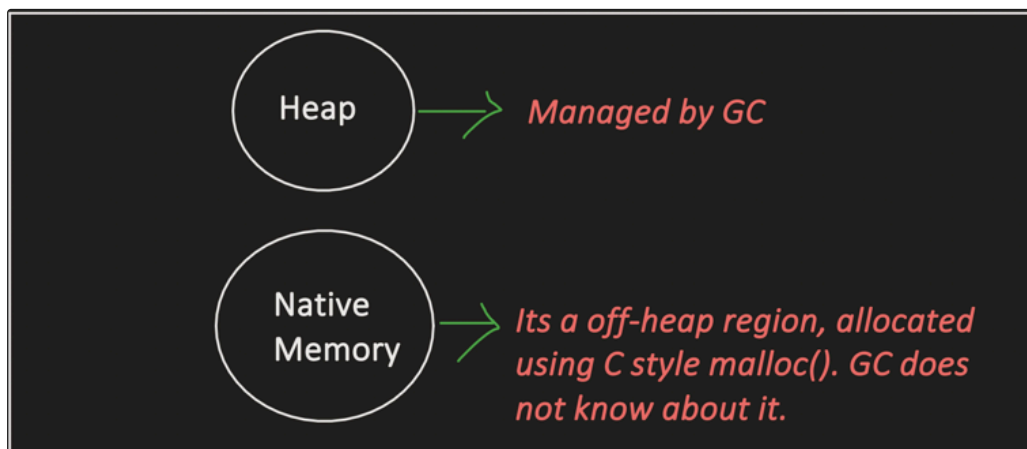
Traditional File handling(Streams):

- With Stream, data is read sequentially.
- `fis.read();` //moves forward only

NIO (Channel):

- With Channel, we can jump at any specific position and start read from there.

3. Double Copying problem:



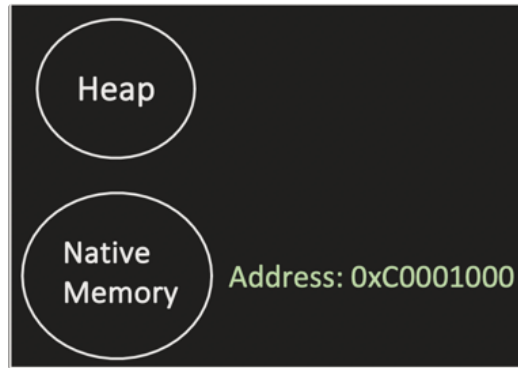
Traditional File Handling(Stream):

```
1 | BufferedInputStream bis = new BufferedInputStream (new
   | FileInputStream("sample.txt"));
2 | while ((byteValue = bis.read()) != -1) {
3 |     //doing something
4 | }
5 |
```

- Internally *BufferedInputStream* creates and manage the Buffer.
- And that Buffer is stored in Heap memory.

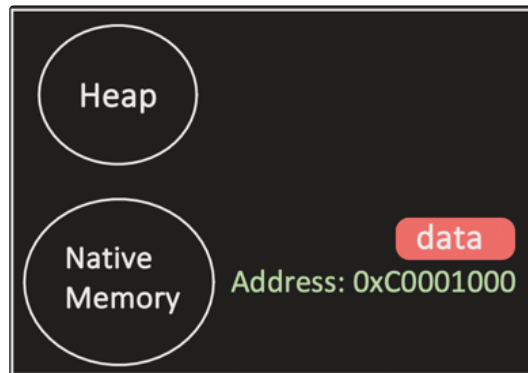
But internally what happens:

JVM - Allocates a temporary space in Native memory.

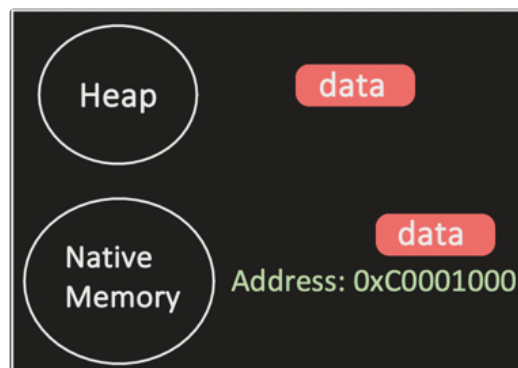


JVM - Ask OS to write the file data at Address 0xC0001000

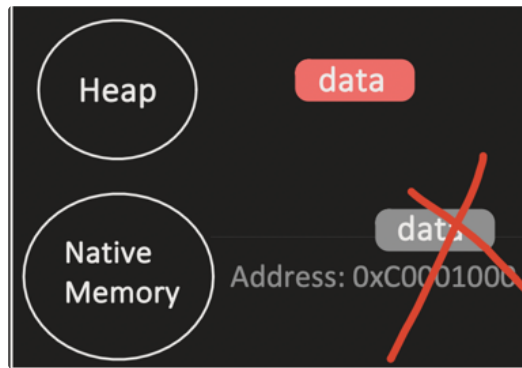
OS - Writes at



JVM - Copies from Native memory to Heap



JVM - Discard Temporary Native memory



So, 2 times File data is copied:

1. From OS to Native memory
2. From Native Memory to Heap memory

But question might be coming, that why can't OS write File data directly to Heap memory address.

Say, when OS is writing the bytes at buffer HEAP address and if GC runs that time.

Garbage Collector can move objects in Heap memory, so their address location can change, so OS can not safely writes into heap memory.

Heap buffer Address = 0x7F3A2B00

OS starts copying the disk data to memory address 0x7F3A2B00

GS runs and moves Head buffer to = 0x8F3B9C00

OS finished writing to old memory address 0x7F3A2B00

But our buffer address is now = 0x8F3B9C00 and it got nothing.

That's why it need fixed memory address, which do not change and that's where Native memory comes into picture.

I know many engineers might be thinking, when we create object like:

```
User userObj = new User();
```

This object is also stored in Heap and this object also get moved during GC process. So why it do not impact here.

Its because:

In Java we work with references, userObj hold reference not exact heap address.
So when GC moves object, it also update the references to point to new location.

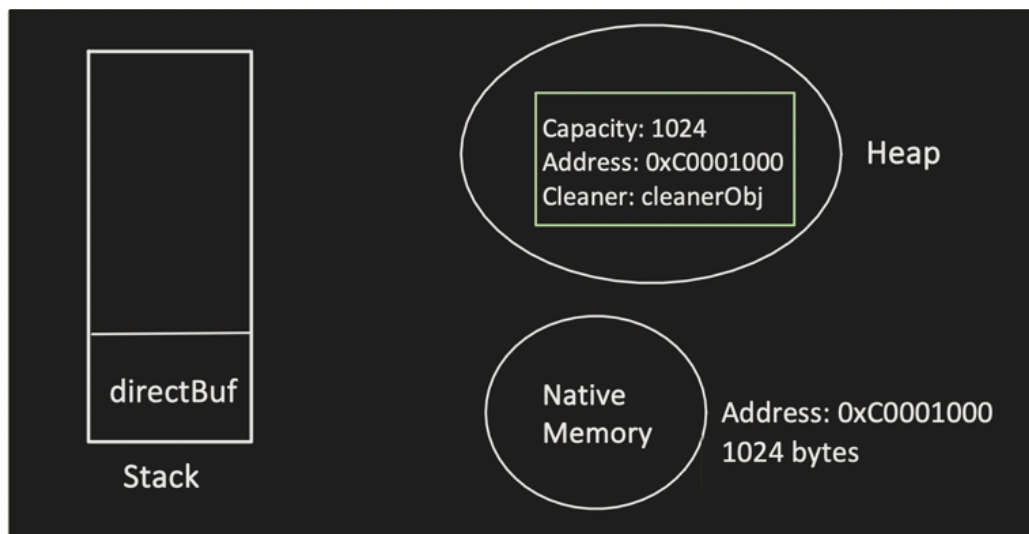
NIO (File IO):

- Buffer is created and managed by us (not by framework internally).
- And we can create the buffer:
 - In Heap memory: same behavior like above (2 times copy, OS → Native and then Native → Heap)
 - In Native memory: only 1 time copy happens (OS → Native memory). Our Code will read from Native memory itself.

Now very important part, who cleans up the Native Memory?

```
ByteBuffer directBuf = ByteBuffer.allocateDirect(1024);
```

allocateDirect - Internally it calls *malloc(1024)* in native memory → gets address *0xC0001000*



Even though buffer memory is allocated in Native memory, but *directBuf* object is stored in heap.

And that is subject to GC.

So when GC runs on this *directBuf* object, it also triggers the Cleaner, which holds Deallocator runnable, which goes and dealloc memory from Address: 0xC0001000

But this has a RISK:

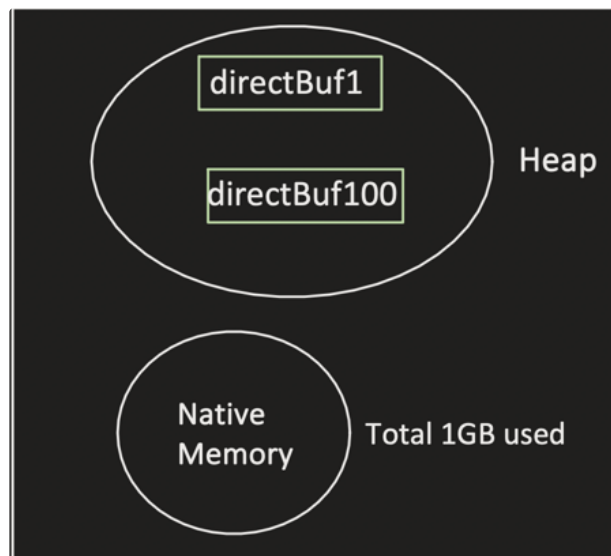
```
ByteBuffer directBuf1 = ByteBuffer.allocateDirect(10MB);
```

.

.

.

```
ByteBuffer directBuf100 = ByteBuffer.allocateDirect(10MB);
```



directBuf1 and directBuf100

1. *Consume very less space,*
2. *Each object with few fields only*

Now lets say buffer objects (directBuf1 . . . directBuf100) are eligible for GC.

But GC decides when to collect. Even though there is no reference of these objects, GC is not bound to run immediately.

GC can only see HEAP memory and it will see HEAP is doing okay, no need to run now.

But Native memory already 1GB is consumed.

And its keep on growing but GC never runs. And soon we will run out of Native memory space.

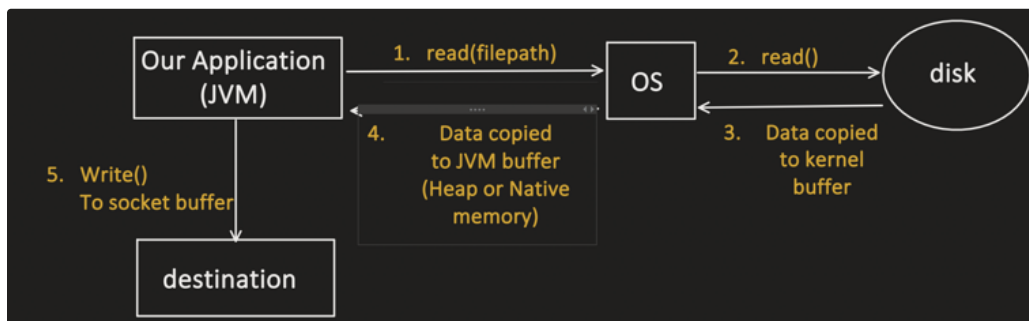
So, my recommendation is when we will create object in Native memory, we should explicitly call cleaner.

`directBuf.cleaner().clean(); // calls free() immediately`

4. Zero-Copy:-

Use case: We have to read the file data from disk and send it to destination.

Traditional File Handling (Stream):

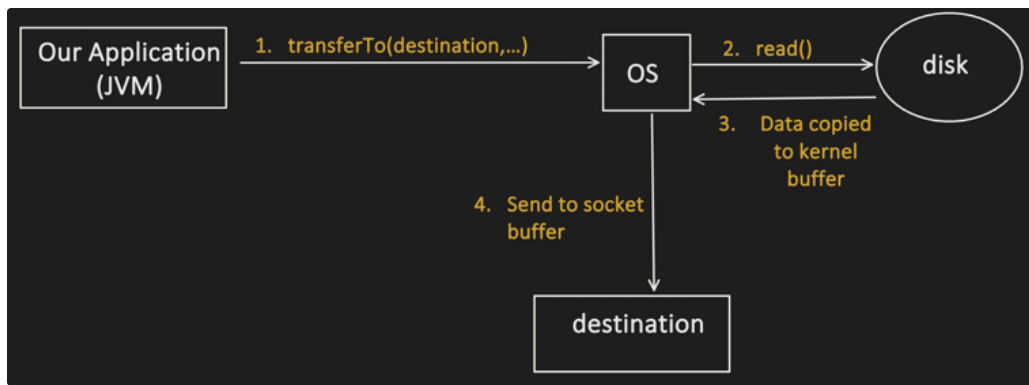


NIO (File IO):

Provides an option of "Zero Copy", means directly from OS it will goes to destination. Our JVM is skipped

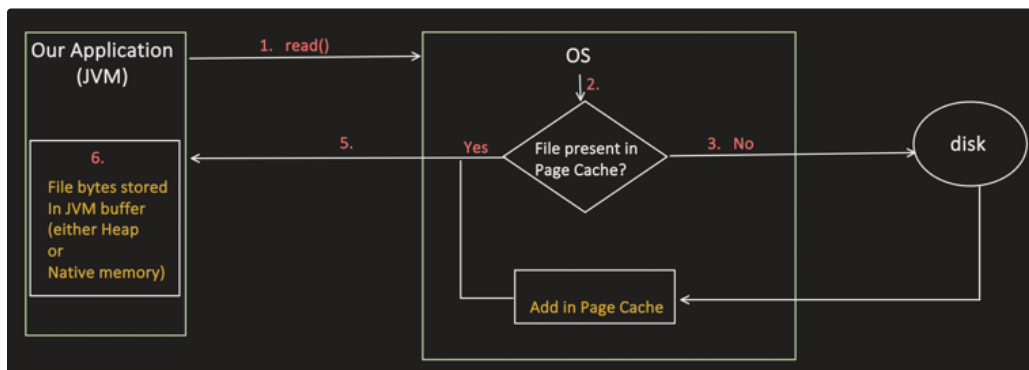
(Zero copy, does not means zero copying of data, but it means that our JVM is bypassed)

Note: This is what KAFKA uses while transferring data from its log file to Consumer. And this is one of the reason which makes Kafka fast.



5. Memory Mapping

Traditional File Handling (Stream): Need to read the bytes from file:

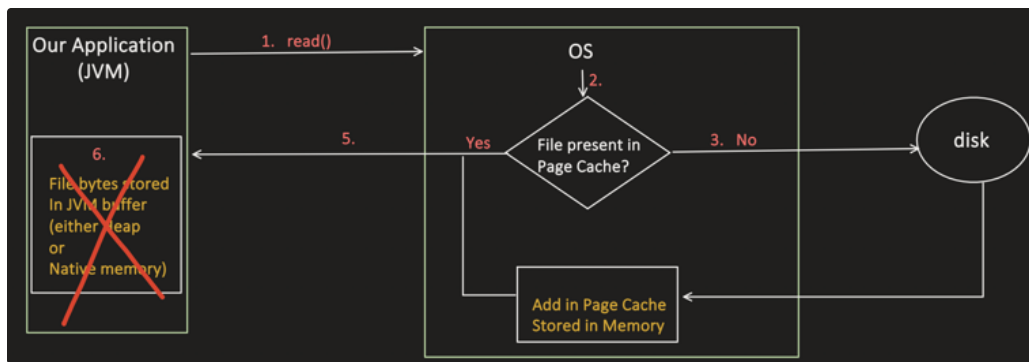


Problem: Same byte of the file is stored at 2 places:

1. Page Cache of OS
2. JVM buffer (either Heap or Native memory)

NIO Memory Mapping feature:

- Aim is to remove this duplicate.
- With Memory Mapping feature of NIO, JVM does not need to maintain the data itself.



This helps in:

- Saving Space (RAM).
- Saving CPU cycles (which is required to copy the data)

But question is, how its done?

RAM: considered it as a array of bytes.

Physical Address	byte
0x0000	
0x0001	
...	...
0xA700	
...	...
0xF600	
...	...

No process, directly works with Physical address.

Why because?

Process 1 → writing at 0x0001

Process 2 → also writing at 0x0002

CLASH. They will overwrite each other.

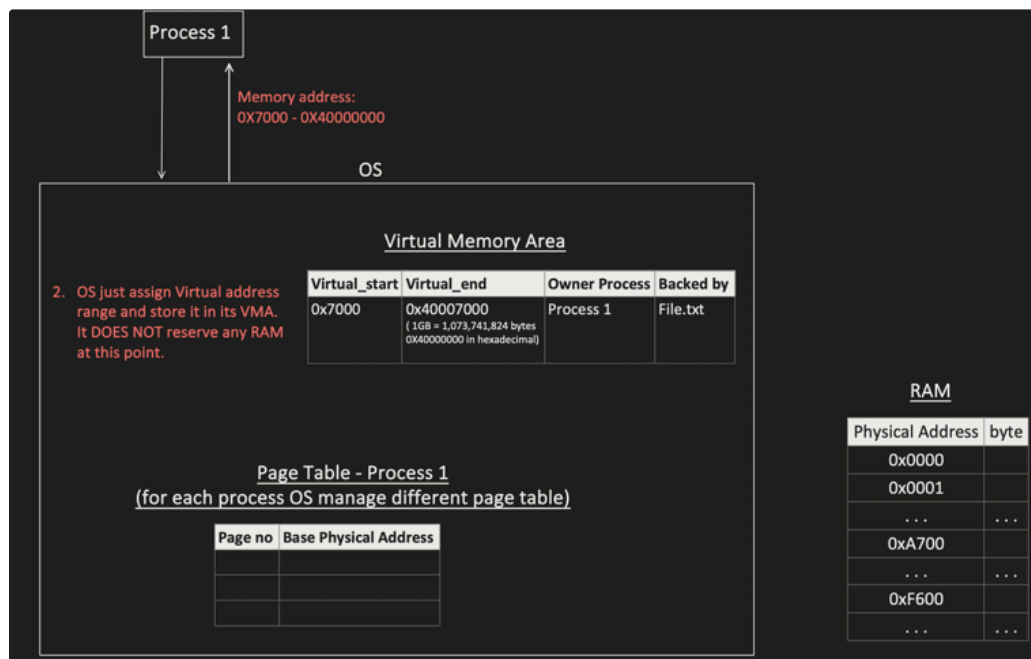
To protect one process from other, concept of virtual address exists.

In that each process gets a chunk of RAM and virtual address mapping is done and stored in Page Table (for each process, OS maintain a Page table).

Ex:

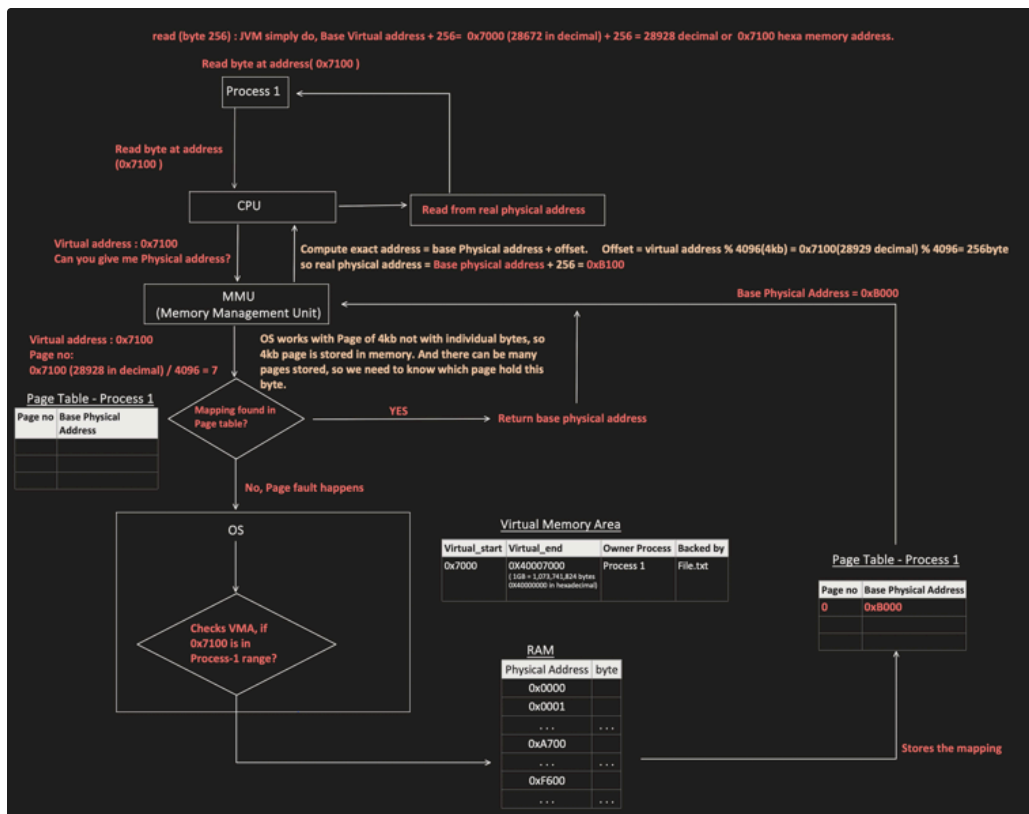
Process can request memory allocation for different purposes and at different times.

say it ask for 1GB memory for storing File data in memory (*NIO, Memory Map scenario: FileChannel.map()*)



- At this point, Page Table is empty, No memory is reserved in RAM. Only Virtual Address range is store in VMA and given to Process.
- Each Process gets a Range of Virtual Address, say: 0x7000 - 0x40007000. And process does not know whether it's a Virtual address or Physical address.

Now lets say process is trying to read file data:



Yes,

Now OS has to read the data from file.

OS know the file path, but it need to know which byte we are interested in.

So it does simple calculation:

0x7100 (trying to read, 28928 in decimal) - 0x7000 (28672 in decimal , virtual base address)

= 256 byte from the file.

But as I mentioned, OS always work with 4KB size page, it do not work with byte individually. So it need to know which 4KB to read (so that this 256th byte comes with that)

So OS does:

0-4095 = Page 0

4096 - 8191 = Page 1

Page = 256/4096 = 0

so OS will try to read Page 0 of file.txt

File system → check where does Page 0 of file.txt stored and return it.

OS finds the space in RAM and stores the page say at base address: 0xB000

So, with above flow, JVM do not need to store the FILE data in its Heap or Native memory. All it stores is virtual address. And rest is all done.

Also OS is only involved when MMU do not find entry (Page fault) in Process Page table. So context switching reduces a lot.